Hindi Vidya Prachar Samiti's

Ramniranjan Jhunjhunwala College of Arts, Science & Commerce

(Empowered Autonomous College)

Affiliated to
UNIVERSITY OF MUMBAI

DEPARTMENT OF INFORMATION TECHNOLOGY
2025 - 2026

M.Sc. (IT) PART 2 - SEM IV

RJSPITE402P – Natural Language Processing

Name: Vishwakarma Shivam Suresh Sushila
Roll No: 6709

Hindi Vidya Prachar Samiti's

Ramniranjan Jhunjhunwala College of Arts, Science & Commerce

(Empowered Autonomous College)

Certificate

This is to certify that Mr. Vishwakarma Shivam Suresh Sushila Roll No 6709 of M.Sc.(I.T.) Part-2 class has completed the required number of experiments in the subject of Natural Language Processing in the Department of Information Technology during the academic year 2025 - 2026 .

Professor In-Charge
Prof. Bharati Bhole

Co-ordinator of IT Department
Prof. Bharati Bhole

College Seal & Date

Examiner

INDEX

Practical No	Title	Page No.
1	**Text Tokenization** a. Tokenization using python split( )function. b. Tokenization using RegEx. c. Tokenization using NLTK/ Tokenization using spacy. d. Tokenization using keras/ Tokenization using Gensim.	1
2	**Corpus** a. Study of various Corpus – Brown, Inaugural, Reuters etc. with various methods like fields, raw, words, sents, categories etc. b. Create and use your own corpora(plaintext, categorical). c. Study Conditional frequency distributions	11
3	**Lemmatization, CFG and Morphemes.** a. Study of Wordnet Dictionary with methods as synsets, definitions, examples, antonyms. b. Write a python program to perform lemmatization. c. Define a grammar(CFG) and generate parse tree results using NLTK. d. Using a porter stemmer library derives derivational and inflectional morphemes.	26
4	**Illustrate part of speech tagging.** a. Part of speech Tagging and chunking of user defined text. b. Named Entity recognition of user defined text. c. Named Entity recognition with diagram using NLTK corpus – treebank.	34
5	**Stop Words.** a. Handling stopwords. b. Using NLTK Adding or Removing Stop Words in NLTK's Default Stop Word. c. Using Spacy Adding and Removing Stop Words in Default Spacy Stop Words List	38
6	**Hyponyms and Hypernyms.** a. Write a program using python to find synonyms and antonyms	44

	of word "active" using Wordnet. b. Compare two nouns. c. Study lemmas, hyponyms, hypernyms	
7	**Tagged Corpora** a. Study of tagged corpora with methods like tagged_sents, tagged_words. b. Write a program to find the most frequent noun tags. c. Map Words to Properties Using Python Dictionaries.	50
8	**NLP libraries for Hindi language.** a. Word tokenization in Hindi.	55

Practical 1: Text Tokenization

---

1. Tokenization using python split( ) function.

Write a Python program that performs **word-level tokenization** on a given paragraph using the built-in split() function.

Code (word tokens)

```
text = """Python is a programming language.
It is widely used for data science, AI, and more! - Shivam
Vishwakarma"""

word_tokens = text.split()
word_tokens
```

Output

```
['Python',
 'is',
 'a',
 'programming',
 'language.',
 'It',
 'is',
 'widely',
 'used',
 'for',
 'data',
 'science,',
 'AI,',
 'and',
 'more!',
 '-',
 'Shivam',
 'Vishwakarma']
```

Code (Sentence tokens)

```
sentence__tokens = text.split('.')
sentence__tokens = [ sentence.strip() for sentence in sentence__tokens]
sentence__tokens
```

Output

```
['Python is a programming language',
 'It is widely used for data science, AI, and more! - Shivam Vishwakarma']
```

2. Tokenization using RegEx.

Write a Python program that performs **word-level tokenization** on a given paragraph using the re module (regular expressions).

Note: If error occurs related to downgrade the numpy version like this:

A module that was compiled using NumPy 1.x cannot be run in NumPy 2.2.0 as it may crash. To support both 1.x and 2.x versions of NumPy, modules must be compiled with NumPy 2.0. Some modules may need to be rebuilt instead, e.g. with 'pybind11>=2.12'. If you are a user of the module, the easiest solution will be to downgrade to 'numpy<2' or try to upgrade the affected module. We expect that some modules will need time to support NumPy 2.

Then run this command
pip install "numpy<2"

Code

```
import nltk  # Import the NLTK library
nltk.download("brown")  # Download the Brown corpus for text data

from nltk.corpus import brown  # Import the Brown corpus
from nltk.tag import RegexpTagger  # Import the RegexpTagger for rule-
based tagging

test_sent = brown.sents(categories="news")[0]  # Select the first
sentence from the 'news' category

# Define regular expression tagger with a set of rules for POS tagging
regexp_tagger = RegexpTagger(
    [
        (r"^-?[0-9]+(\.[0-9]+)?$", "CD"),  # Rule for numbers (cardinal
numbers)
        (r"(The|the|A|a|An|an)$", "AT"),  # Rule for determiners
        (r".*able$", "JJ"),  # Rule for adjectives ending in 'able'
        (r".*ness$", "NN"),  # Rule for nouns ending in 'ness'
        (r".*ly$", "RB"),  # Rule for adverbs ending in 'ly'
        (r".*s$", "NNS"),  # Rule for plural nouns
        (r".*ing$", "VBG"),  # Rule for present participles
        (r".*ed$", "VBD"),  # Rule for past tense verbs
```

```
        (r".*", "NN"),  # Default rule for all other words (noun)
    ]
)
print(regexp_tagger)  # Print the RegexpTagger object
tagged_sentence = regexp_tagger.tag(
    test_sent
)  # Tag the sentence with the defined rules
print(tagged_sentence)  # Print the tagged sentence
```

Output

```
<Regexp Tagger: size=9>

[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data]   Package brown is already up-to-date!

[('The', 'AT'),
 ('Fulton', 'NN'),
 ('County', 'NN'),
 ('Grand', 'NN'),
 ('Jury', 'NN'),
 ('said', 'NN'),
 ('Friday', 'NN'),
 ('an', 'AT'),
 ('investigation', 'NN'),
 ('of', 'NN'),
 ("Atlanta's", 'NNS'),
 ('recent', 'NN'),
 ('primary', 'NN'),
 ('election', 'NN'),
 ('produced', 'VBD'),
 ('``', 'NN'),
 ('no', 'NN'),
 ('evidence', 'NN'),
 ("''", 'NN'),
 ('that', 'NN'),
 ('any', 'NN'),
 ('irregularities', 'NNS'),
 ('took', 'NN'),
 ('place', 'NN'),
 ('.', 'NN')]
```

3. Tokenization using NLTK/ Tokenization using spaCy.

Write a Python program that performs **word-level tokenization** on a given paragraph using NLTK / spaCy.

a. Tokenization using NLTK.

Code

```
import nltk
from nltk.tokenize import word_tokenize

# Download tokenizer data (run once)
nltk.download('punkt_tab')

# Input paragraph
paragraph = """Natural Language Processing (NLP) helps computers
understand human language.
Python is widely used for NLP tasks such as tokenization, sentiment
analysis, and translation."""

# Perform word-level tokenization using NLTK
nltk_tokens = word_tokenize(paragraph)

print("NLTK Word Tokens:")
nltk_tokens
```

Output

```
NLTK Word Tokens:
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!

['Natural',
 'Language',
 'Processing',
 '(',
 'NLP',
 ')',
 'helps',
 'computers',
 'understand',
 'human',
 'language',
 '.',
 'Python',
 'is',
 'widely',
 'used',
 'for',
 'NLP',
 'tasks',
 'such',
 'as',
 'tokenization',
 ',',
 'sentiment',
 'analysis',
 ',',
 'and',
 'translation',
 '.']
```

b. Tokenization using the spaCy library

Download en_core_web_sm

!python -m spacy download en_core_web_sm

```
!python -m spacy download en_core_web_sm

Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 81.6 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
```

Code

```
import spacy # import spacy library
# Load the spaCy's pre-trained English model
nlp = spacy.load("en_core_web_sm")

# Define a sentence for analysis
text = "He teaches students walking to school."
doc = nlp(text) # Process the input text with spaCy

# Analyze tokens for morphological information
for token in doc:  # Loop through each token in the processed text
    print(f"Token: {token.text}")  # Print the token text
    print(f" Lemma: {token.lemma_}")  # Print the lemma (base form)
    print(f" POS: {token.pos_}")  # Print the part of speech
    print(f" Morphology: {token.morph}")  # Print morphological features
    print(f" Inflection: {token.tag_}\n")  # Print the inflection tag
```

Output

```
Token: He
 Lemma: he
 POS: PRON
 Morphology: Case=Nom|Gender=Masc|Number=Sing|Person=3|PronType=Prs
 Inflection: PRP

Token: teaches
 Lemma: teach
 POS: VERB
 Morphology: Number=Sing|Person=3|Tense=Pres|VerbForm=Fin
 Inflection: VBZ

Token: students
 Lemma: student
 POS: NOUN
 Morphology: Number=Plur
 Inflection: NNS

Token: walking
 Lemma: walk
 POS: VERB
 Morphology: Aspect=Prog|Tense=Pres|VerbForm=Part
 Inflection: VBG

Token: to
 Lemma: to
 POS: ADP
 Morphology:
 Inflection: IN

Token: school
 Lemma: school
 POS: NOUN
 Morphology: Number=Sing
 Inflection: NN

Token: .
 Lemma: .
 POS: PUNCT
 Morphology: PunctType=Peri
 Inflection: .
```

4. Tokenization using keras/ Tokenization using Gensim.

Write a Python program that performs **word-level tokenization** on a given paragraph using Keras/ Gensim.

Install the following python Libraries:
!pip install tensorflow
!pip install keras
!pip install gensim

a. Tokenization using Keras

Code

```
from tensorflow.keras.preprocessing.text import text_to_word_sequence

# Define a text string
text = """Python is programming language.
It is widely used for data science, AI, and more!
- Shivam Vishwakarma"""

# Tokenize the text into words using Keras tokenizer
tokens = text_to_word_sequence(text)
tokens
```

Output

```
['python',
 'is',
 'programming',
 'language',
 'it',
 'is',
 'widely',
 'used',
 'for',
 'data',
 'science',
 'ai',
 'and',
 'more',
 'shivam',
 'vishwakarma']
```

b. Tokenization using Gensim

Code

```
# Import Gensim's simple tokenizer
from gensim.utils import simple_preprocess

# Define a text string
text = """Python is programming language.
It is widely used for data science, AI, and more!
- Shivam Vishwakarma"""

# Tokenize the text into words using Gensim's simple_preprocess
tokens = simple_preprocess(text)
tokens
```

Output

```
['python',
 'is',
 'programming',
 'language',
 'it',
 'is',
 'widely',
 'used',
 'for',
 'data',
 'science',
 'ai',
 'and',
 'more',
 'shivam',
 'vishwakarma']
```

Practical 2: Corpus

---

1. **Study of various Corpus – Brown, Inaugural, Reuters CORPUS with various methods like fields, raw, words, sents, categories etc.**

- **Study of Brown Corpus**
  Write a Python program using NLTK to explore the Brown Corpus and perform the following tasks:

1. Import the Brown Corpus from NLTK.
2. Display all available categories using categories().
3. Display the file IDs using fileids().
4. Extract and display
   a. The first 200 characters using raw()
   b. The first 20 words using words()
   c. The first 5 sentences using sents()
5. Display words from a specific category (e.g., "news").
6. Count the total number of words in a selected category.

Code

```
# 1. Import the Brown Corpus from NLTK.
import nltk
from nltk.corpus import brown
nltk.download('brown')
print("\nBrown Corpus imported from NLTK")

# 2. Display all available categories
print("\nBrown Corpus Categories: ")
print(brown.categories())

# 3. Display file IDs
print("\nBrown Corpus File IDs ")
print(brown.fileids())

# 4. a. Extract first 200 characters using raw()
print("\nFirst 200 Characters (raw text) ")
print(brown.raw()[:200])

# 4. b. Extract first 20 words
```

```
print("\nFirst 20 Words ")
print(brown.words()[:20])

# 4. c. Extract first 5 sentences
print("\nFirst 5 Sentences ")
for sent in brown.sents()[:5]:
    print(sent)

# 5. Display words from a specific category ("news")
print("\nWords from 'news' category (first 20) ")
print(brown.words(categories='news')[:20])

# 6. Count total number of words in selected category
total_words_news = len(brown.words(categories='news'))

print("\nTotal number of words in 'news' category:")
print(total_words_news)
```

Output

```
Brown Corpus imported from NLTK

Brown Corpus Categories:
['adventure', 'belles_lettres', 'editorial', 'fiction', 'government', 'hobbies', 'humor', 'learned', 'le

Brown Corpus File IDs
['ca01', 'ca02', 'ca03', 'ca04', 'ca05', 'ca06', 'ca07', 'ca08', 'ca09', 'ca10', 'ca11', 'ca12', 'ca13'

First 200 Characters (raw text)


        The/at Fulton/np-tl County/nn-tl Grand/jj-tl Jury/nn-tl said/vbd Friday/nr an/at investigation/

First 20 Words
['The', 'Fulton', 'County', 'Grand', 'Jury', 'said', 'Friday', 'an', 'investigation', 'of', "Atlanta's"

First 5 Sentences
['The', 'Fulton', 'County', 'Grand', 'Jury', 'said', 'Friday', 'an', 'investigation', 'of', "Atlanta's"
['The', 'jury', 'further', 'said', 'in', 'term-end', 'presentments', 'that', 'the', 'City', 'Executive'
['The', 'September-October', 'term', 'jury', 'had', 'been', 'charged', 'by', 'Fulton', 'Superior', 'Cou
['``', 'Only', 'a', 'relative', 'handful', 'of', 'such', 'reports', 'was', 'received', "''", ',', 'the'
['The', 'jury', 'said', 'it', 'did', 'find', 'that', 'many', 'of', "Georgia's", 'registration', 'and',

Words from 'news' category (first 20)
['The', 'Fulton', 'County', 'Grand', 'Jury', 'said', 'Friday', 'an', 'investigation', 'of', "Atlanta's"

Total number of words in 'news' category:
100554
```

- **Study of Inaugural Address Corpus**

  Write a Python program to analyze the corpus using various corpus access methods

1. Import the Inaugural Corpus from NLTK.
2. Display all available file IDs using fileids().
3. Extract:
   a. The first 300 characters using raw()
   b. The first 30 words using words()
   c. The first 5 sentences using sents()
4. Find how many times a specific word (e.g., "freedom") appears in a selected speech.
5. Compare word frequency between two different inaugural speeches

Code

```
# 1. Import the Inaugural Corpus from NLTK.
import nltk
from nltk.corpus import inaugural
nltk.download('inaugural')
nltk.download('punkt_tab')
print("\nInaugural Corpus imported from NLTK")

# 2. Display all available file IDs
print("\nInaugural Corpus File IDs ")
print(inaugural.fileids())

# 3. a. Extract first 300 characters using raw()
print("\nFirst 300 Characters (raw text) ")
print(inaugural.raw()[:300])

# 3. b. Extract first 30 words
print("\nFirst 30 Words ")
print(inaugural.words()[:30])

# 3. c. Extract first 5 sentences
print("\nFirst 5 Sentences ")
for sent in inaugural.sents()[:5]:
    print(sent)

# 4. Find how many times a specific word appears in a selected speech
selected_speech = '2009-Obama.txt'
word_to_find = 'freedom'
```

```
speech_words = [w.lower() for w in
inaugural.words(fileids=selected_speech)]
word_count = speech_words.count(word_to_find)

print("\nNumber of times the word appears in selected speech ")
print(selected_speech, ":", word_to_find, "=", word_count)

# 5. Compare word frequency between two different inaugural speeches
speech1 = '2009-Obama.txt'
speech2 = '1961-Kennedy.txt'

words1 = [w.lower() for w in inaugural.words(fileids=speech1)]
words2 = [w.lower() for w in inaugural.words(fileids=speech2)]

freq1 = words1.count(word_to_find)
freq2 = words2.count(word_to_find)

print("\nWord frequency comparison ")
print(speech1, ":", word_to_find, freq1)
print(speech2, ":", word_to_find, freq2)
```

Output

```
Inaugural Corpus imported from NLTK

Inaugural Corpus File IDs
['1789-Washington.txt', '1793-Washington.txt', '1797-Adams.txt', '1801-Jefferson.txt', '1805-Jefferson.txt'

First 300 Characters (raw text)
Fellow-Citizens of the Senate and of the House of Representatives:

Among the vicissitudes incident to life no event could have filled me with greater anxieties than that of w

First 30 Words
['Fellow', '-', 'Citizens', 'of', 'the', 'Senate', 'and', 'of', 'the', 'House', 'of', 'Representatives', ':

First 5 Sentences
['Fellow', '-', 'Citizens', 'of', 'the', 'Senate', 'and', 'of', 'the', 'House', 'of', 'Representatives', ':
['Among', 'the', 'vicissitudes', 'incident', 'to', 'life', 'no', 'event', 'could', 'have', 'filled', 'me',
['On', 'the', 'one', 'hand', ',', 'I', 'was', 'summoned', 'by', 'my', 'Country', ',', 'whose', 'voice', 'I'
['On', 'the', 'other', 'hand', ',', 'the', 'magnitude', 'and', 'difficulty', 'of', 'the', 'trust', 'to', 'w
['In', 'this', 'conflict', 'of', 'emotions', 'all', 'I', 'dare', 'aver', 'is', 'that', 'it', 'has', 'been',

Number of times the word appears in selected speech
2009-Obama.txt : freedom = 3

Word frequency comparison
2009-Obama.txt : freedom 3
1961-Kennedy.txt : freedom 4
```

- **Study of Reuters Corpus**

  Write a Python program to explore the corpus structure and analyze its categories.

1. Import the Reuters Corpus from NLTK.
2. Display all available categories using categories().
3. Display all file IDs using fileids().
4. Extract:
   a. Raw text using raw()
   b. Word tokens using words()
   c. Sentence tokens using sents()
5. Display file IDs belonging to a specific category (e.g., "trade").
6. Count the number of documents under a selected category.
7. Find the most frequent words in a chosen category.

Code

```
import nltk # 1. Import the Reuters Corpus from NLTK.
from nltk.corpus import reuters
from nltk import FreqDist
nltk.download('reuters')
nltk.download('punkt')

print("\nReuters Corpus imported from NLTK")

# 2. Display all available categories
print("\nReuters Corpus Categories ")
print(reuters.categories())

# 3. Display all file IDs
print("\nReuters Corpus File IDs ")
print(reuters.fileids())

print("\nRaw Text (first 300 characters) ")
print(reuters.raw()[:300]) # 4. a. Extract raw text
print("\nFirst 30 Word Tokens ")
print(reuters.words()[:30]) # 4. b. Extract word tokens

# 4. c. Extract sentence tokens
print("\nFirst 5 Sentences ")
for sent in reuters.sents()[:5]:
    print(sent)
```

```python
# 5. Display file IDs belonging to a specific category ("trade")
category_name = "trade"
print("\nFile IDs in selected category ")
print(reuters.fileids(category_name))

# 6. Count number of documents under selected category
doc_count = len(reuters.fileids(category_name))
print("\nNumber of documents in selected category ")
print(doc_count)

# 7. Find most frequent words in chosen category
words_in_category = [w.lower() for w in
reuters.words(categories=category_name)]
freq_dist = FreqDist(words_in_category)
print("\nMost frequent words in selected category ")
print(freq_dist.most_common(20))
```

Output

```
Reuters Corpus imported from NLTK

Reuters Corpus Categories
['acq', 'alum', 'barley', 'bop', 'carcass', 'castor-oil', 'cocoa', 'coconut', 'coconut-oil', 'coffee', 'copper'

Reuters Corpus File IDs
['test/14826', 'test/14828', 'test/14829', 'test/14832', 'test/14833', 'test/14839', 'test/14840', 'test/14841'

Raw Text (first 300 characters)
ASIAN EXPORTERS FEAR DAMAGE FROM U.S.-JAPAN RIFT
  Mounting trade friction between the
  U.S. And Japan has raised fears among many of Asia's exporting
  nations that the row could inflict far-reaching economic
  damage, businessmen and officials said.
      They told Reuter correspondents in Asian

First 30 Word Tokens
['ASIAN', 'EXPORTERS', 'FEAR', 'DAMAGE', 'FROM', 'U', '.', 'S', '.-', 'JAPAN', 'RIFT', 'Mounting', 'trade', 'fr

First 5 Sentences
['ASIAN', 'EXPORTERS', 'FEAR', 'DAMAGE', 'FROM', 'U', '.', 'S', '.-', 'JAPAN', 'RIFT', 'Mounting', 'trade', 'fr
['They', 'told', 'Reuter', 'correspondents', 'in', 'Asian', 'capitals', 'a', 'U', '.', 'S', '.', 'Move', 'agains
['But', 'some', 'exporters', 'said', 'that', 'while', 'the', 'conflict', 'would', 'hurt', 'them', 'in', 'the',
['The', 'U', '.', 'S', '.', 'Has', 'said', 'it', 'will', 'impose', '300', 'mln', 'dlrs', 'of', 'tariffs', 'on',
['Unofficial', 'Japanese', 'estimates', 'put', 'the', 'impact', 'of', 'the', 'tariffs', 'at', '10', 'billion',

File IDs in selected category
['test/14826', 'test/14832', 'test/14858', 'test/14862', 'test/14881', 'test/14904', 'test/14912', 'test/15154'

Number of documents in selected category
485

Most frequent words in selected category
[('.', 8440), ('the', 7177), (',', 4280), ('to', 4270), ('of', 2980), ('in', 2967), ('and', 2520), ('s', 2449),
```

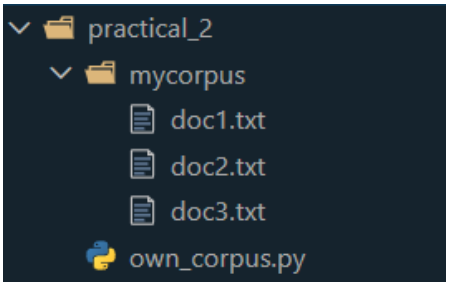
2. Create and use your own corpora(plaintext, categorical).

i. Creating and Using Your Own Plaintext Corpus

Create your own **plaintext corpus** consisting of multiple .txt files stored inside a folder and write a Python program using NLTK to load and analyze the corpus.

Tasks:

1. Create a folder named mycorpus.
2. Add at least 3 text files (e.g., doc1.txt, doc2.txt, doc3.txt) containing sample text.



3. Use NLTK's PlaintextCorpusReader to load the corpus.
4. Display:
   a. All file IDs using fileids()
   b. Raw content of a selected file using raw()
   c. Word tokens using words()
   d. Sentence tokens using sents()
5. Count the total number of:
   a. Files
   b. Words
   c. Sentences
6. Display the most frequent 10 words in the corpus.

Code

```
# Import required libraries
import nltk
from nltk.corpus import PlaintextCorpusReader
from nltk import FreqDist
nltk.download('punkt')

# Create a folder named 'mycorpus' and place doc1.txt,.... inside it
# Define the corpus root folder
corpus_root = '/content/sample_data/mycorpus' # For Google Colab
# corpus_root = './mycorpus' # For local system
```

```
# 3. Load the corpus using PlaintextCorpusReader
my_corpus = PlaintextCorpusReader(corpus_root, '.*\.txt')

# 4. a. Display all file IDs
print("\nFile IDs in Corpus ")
print(my_corpus.fileids())

# 4. b. Display raw content of a selected file
selected_file = 'doc1.txt'
print("\nRaw Content of Selected File ")
print(my_corpus.raw(selected_file))

# 4. c. Display word tokens
print("\nFirst 30 Word Tokens ")
print(my_corpus.words()[:30])

# 4. d. Display sentence tokens
print("\nFirst 5 Sentences ")
for sent in my_corpus.sents()[:5]:
    print(sent)

# 5. a. Count total number of files
total_files = len(my_corpus.fileids())
print("\nTotal Number of Files: ", total_files)

# 5. b. Count total number of words
total_words = len(my_corpus.words())
print("\nTotal Number of Words ", total_words)

# 5. c. Count total number of sentences
total_sentences = len(my_corpus.sents())
print("\nTotal Number of Sentences ", total_sentences)

# 6. Display most frequent 10 words
freq_dist = FreqDist([w.lower() for w in my_corpus.words()])
print("\nMost Frequent 10 Words ")
print(freq_dist.most_common(10))
```

Output

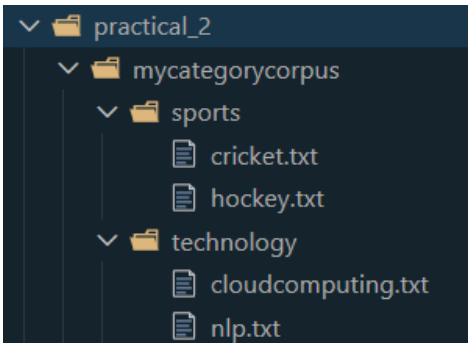
```
Plaintext Corpus Example

File IDs in Corpus
['doc1.txt', 'doc2.txt', 'doc3.txt']

Raw Content of Selected File
Virat Kohli[a] (born 5 November 1988) is an Indian international cricketer and the former all-format captain of the I

Kohli was the captain of the 2008 U19 World Cup winning team and was a crucial member of the teams that won 2011 ODI

First 30 Word Tokens
['Virat', 'Kohli', '[', 'a', ']', '(', 'born', '5', 'November', '1988', ')', 'is', 'an', 'Indian', 'international', '

First 5 Sentences
['Virat', 'Kohli', '[', 'a', ']', '(', 'born', '5', 'November', '1988', ')', 'is', 'an', 'Indian', 'international', '
['[', '3', ']', 'He', 'is', 'a', 'right', '-', 'handed', 'batter', 'and', 'occasional', 'right', '-', 'arm', 'medium'
['Considered', 'one', 'of', 'the', 'greatest', 'all', '-', 'format', 'batsmen', 'in', 'the', 'history', 'of', 'cricke
['[', '4', ']', 'Kohli', 'has', 'the', 'most', 'centuries', 'in', 'ODIs', 'and', 'the', 'second', '-', 'most', 'centu
['He', 'is', 'also', 'the', 'leading', 'run', '-', 'scorer', 'in', 'the', 'Indian', 'Premier', 'League', '.']

Total Number of Files
3

Total Number of Words
969

Total Number of Sentences
40

Most Frequent 10 Words
[('the', 47), (',', 38), ('.', 36), ('and', 28), ('in', 24), ('-', 19), ('of', 18), ('(', 14), ('s', 14), ('"', 14)]
```

ii. Creating and Using Your Own Categorical Corpus

Create your own **categorical corpus** with multiple folders representing categories and write a Python program using NLTK to analyze it.

Tasks:

1. Create a main folder named mycategorycorpus.
2. Inside it, create at least two category folders (e.g., sports, technology).
3. Add at least 2 text files in each category.



4. Use NLTK's CategorizedPlaintextCorpusReader to load the corpus.
5. Display:
   a. All categories using categories()
   b. File IDs using fileids()
   c. Files belonging to a specific category
6. Extract:
   a. Words from a selected category
   b. Sentences from a selected category
7. Compare the total word count between categories.
8. Identify the most frequent words in each category.

Code

```
# 1. Import required libraries
import nltk
from nltk.corpus import CategorizedPlaintextCorpusReader
from nltk import FreqDist

nltk.download('punkt')

print("\nCategorized Plaintext Corpus Example")

# Define corpus root folder
corpus_root = '/content/sample_data/mycategorycorpus' # for Google Colab
```

```
# corpus_root = './mycorpus' # For local system

# 4. Load categorized corpus
my_corpus = CategorizedPlaintextCorpusReader(
    corpus_root,
    r'.*\.txt',
    cat_pattern=r'(.+)/.*'
)

# 5. a. Display all categories
print("\nCategories ")
print(my_corpus.categories())

# 5. b. Display file IDs
print("\nFile IDs ")
print(my_corpus.fileids())

# 5. c. Display files belonging to a specific category
selected_category = 'sports'

print("\nFiles in selected category ")
print(my_corpus.fileids(categories=selected_category))

# 6. a. Extract words from selected category
print("\nFirst 30 words from selected category ")
print(my_corpus.words(categories=selected_category)[:30])

# 6. b. Extract sentences from selected category
print("\nFirst 5 sentences from selected category ")
for sent in my_corpus.sents(categories=selected_category)[:5]:
    print(sent)

# 7. Compare total word count between categories
print("\nTotal word count per category ")
for cat in my_corpus.categories():
    total_words = len(my_corpus.words(categories=cat))
    print(cat, total_words)

# 8. Most frequent words in each category
```

```
print("\nMost frequent words in each category ")
for cat in my_corpus.categories():
    words = [w.lower() for w in my_corpus.words(categories=cat)]
    freq = FreqDist(words)
    print("\nCategory:", cat)
    print(freq.most_common(10))
```

Output

```
Categorized Plaintext Corpus Example

Categories
['sports', 'technology']

File IDs
['sports/cricket.txt', 'sports/hockey.txt', 'technology/cloudcomputing.txt', 'technology/nlp.txt']

Files in selected category
['sports/cricket.txt', 'sports/hockey.txt']

First 30 words from selected category
['Cricket', 'is', 'a', 'bat', '-', 'and', '-', 'ball', 'game', 'that', 'is', 'played', 'between', 'two', 'teams'

First 5 sentences from selected category
['Cricket', 'is', 'a', 'bat', '-', 'and', '-', 'ball', 'game', 'that', 'is', 'played', 'between', 'two', 'teams'
['Two', 'players', 'from', 'the', 'batting', 'team', ',', 'the', 'striker', 'and', 'nonstriker', ',', 'stand', '
['The', 'striker', "'", 's', 'goal', 'is', 'to', 'hit', 'the', 'bowled', 'ball', 'with', 'the', 'bat', 'and', 't
['Runs', 'are', 'also', 'scored', 'when', 'the', 'ball', 'reaches', 'the', 'boundary', 'of', 'the', 'field', 'or
['Hockey', 'is', 'a', 'family', 'of', 'stick', 'sports', 'where', 'two', 'opposing', 'teams', 'use', 'hockey', '

Total word count per category
sports 293
technology 106

Most frequent words in each category

Category: sports
[(',', 22), ('the', 21), ('of', 11), ('hockey', 11), ('.', 9), ('a', 8), ('is', 6), ('and', 6), ('ball', 6), ('-

Category: technology
[('is', 6), ('a', 4), ('and', 4), ('the', 3), ('"', 3), ('to', 3), ('of', 3), ('nlp', 3), ('.', 3), (',', 3)]
```

3. Study Conditional frequency distributions.

Write a Python program using NLTK to study and analyze **Conditional Frequency Distributions** using a standard corpus such as the Brown Corpus or Inaugural Address Corpus.

Tasks

1. Import the required libraries (nltk and ConditionalFreqDist).
2. Load a corpus (e.g., Brown Corpus).
3. Select at least two categories (e.g., news and romance).
4. Create a Conditional Frequency Distribution for selected words such as:
    a. "the"
    b. "and"
    c. "government"
    d. "love"
5. Display:
    a. Frequency count of selected words across categories.
    b. Tabular comparison using cfd.tabulate().
6. Plot the distribution using cfd.plot().
7. Interpret the results and explain differences in word usage between categories.

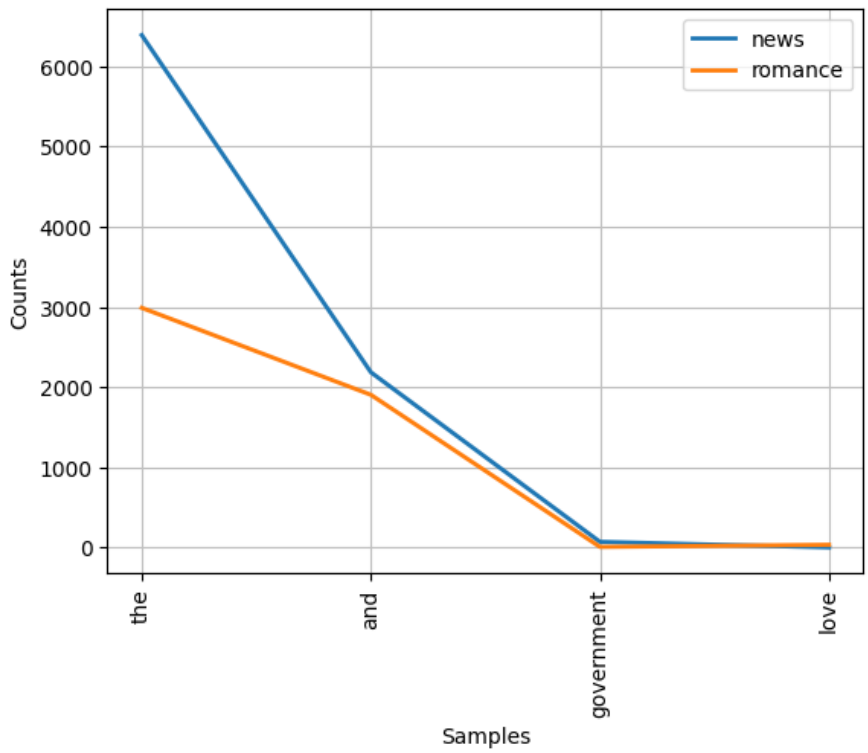
Code

```
# 1. Import required libraries
import nltk
from nltk.corpus import brown
from nltk.probability import ConditionalFreqDist

# 2. Download required corpus
nltk.download('brown')

# 3. Select categories
categories = ['news', 'romance']

# 4. Select target words
target_words = ['the', 'and', 'government', 'love']

# Create Conditional Frequency Distribution
cfd = ConditionalFreqDist(
    (category, word.lower())
    for category in categories
    for word in brown.words(categories=category)
```

```
    if word.lower() in target_words
)

# 5. a. Display frequency count of selected words across categories
print("\nFrequency counts of selected words")
for category in categories:
    print("\nCategory:", category)
    for word in target_words:
        print(word, ":", cfd[category][word])

# 5. b. Tabular comparison
print("\nTabular Comparison")
cfd.tabulate(conditions=categories, samples=target_words)

# 6. Plot the distribution
cfd.plot(conditions=categories, samples=target_words)

# 7. Interpretation
```

Output

```
Frequency counts of selected words

Category: news
the : 6386
and : 2186
government : 73
love : 3

Category: romance
the : 2988
and : 1905
government : 10
love : 36

Tabular Comparison
               the        and government       love
   news       6386       2186         73          3
romance       2988       1905         10         36
```



Interpretation

- The news category is expected to contain more formal and political terms like 'government'.
- Romance categories may contain more emotional or relational terms like 'love'.
- Common words such as 'the' and 'and' appear frequently in both categories.

Practical 3: Lemmatization, CFG and Morphemes.

---

1. **Study of Wordnet Dictionary with methods as synsets, definitions, examples, antonyms.**

Code

```
import nltk
nltk.download('wordnet') # Download datasets
from nltk.corpus import wordnet

print("WORDNET DICTIONARY STUDY")

print("\n1. Synsets (Meanings) of the word 'computer':")
print(wordnet.synsets("computer"))

print("\n2. Definition of 'computer.n.01':")
print(wordnet.synset("computer.n.01").definition())

print("\n3. Example sentences of 'computer.n.01':")
print(wordnet.synset("computer.n.01").examples())

print("\n4. Antonyms of the verb 'buy' (buy.v.01):")
print(wordnet.lemma("buy.v.01.buy").antonyms())

print("\n5. Synsets (Meanings) of the word 'man':")
print(wordnet.synsets("man"))

print("\n6. Definition of 'valet.n.01':")
print(wordnet.synset("valet.n.01").definition())

print("\n7. Example sentences of 'man.n.03':")
print(wordnet.synset("man.n.03").examples())

print("\n8. Antonyms of the noun 'man' (man.n.01):")
print(wordnet.lemma("man.n.01.man").antonyms())
```

Output

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
WORDNET DICTIONARY STUDY

1. Synsets (Meanings) of the word 'computer':
[Synset('computer.n.01'), Synset('calculator.n.01')]

2. Definition of 'computer.n.01':
a machine for performing calculations automatically

3. Example sentences of 'computer.n.01':
[]

4. Antonyms of the verb 'buy' (buy.v.01):
[Lemma('sell.v.01.sell')]

5. Synsets (Meanings) of the word 'man':
[Synset('man.n.01'), Synset('serviceman.n.01'), Synset('man.n.03'),
Synset('homo.n.02'), Synset('man.n.05'), Synset('man.n.06'),
Synset('valet.n.01'), Synset('man.n.08'), Synset('man.n.09'),
Synset('man.n.10'), Synset('world.n.08'), Synset('man.v.01'),
Synset('man.v.02')]

6. Definition of 'valet.n.01':
a manservant who acts as a personal attendant to his employer

7. Example sentences of 'man.n.03':
['it was every man for himself']

8. Antonyms of the noun 'man' (man.n.01):
[Lemma('woman.n.01.woman')]
```

2. **Write a python program to perform lemmatization.**

Code

```
import nltk
from nltk.stem import WordNetLemmatizer

# Download required datasets
nltk.download('wordnet')
nltk.download('punkt_tab')

# Function to perform lemmatization
def lemmatize_text(text):
    lemmatizer = WordNetLemmatizer()   # Initialize lemmatizer
    tokens = nltk.tokenize.word_tokenize(text)   # Tokenize text
    # Lemmatize each word (verb form)
    lemmatized_text = ' '.join([lemmatizer.lemmatize(word, pos='v') for
word in tokens])
    return lemmatized_text

# First example
text = "Students are studying different subjects and preparing for their
examinations."
result = lemmatize_text(text)

print("\nOriginal text:", text)
print("Lemmatized text:", result)

# Second example
lemmatizer = WordNetLemmatizer()

string = "The engineers were designing systems, testing applications,
and fixing bugs."

words = nltk.tokenize.word_tokenize(string)

lemmatized_words = [lemmatizer.lemmatize(word, pos='v') for word in
words]

print("Lemmatized words:", lemmatized_words)
```

Output

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.

Original text: Students are studying different subjects and preparing for
their examinations.

Lemmatized text: Students be study different subject and prepare for their
examinations .


Lemmatized words: ['The', 'engineer', 'be', 'design', 'systems', ',',
'test', 'applications', ',', 'and', 'fix', 'bug', '.']
```

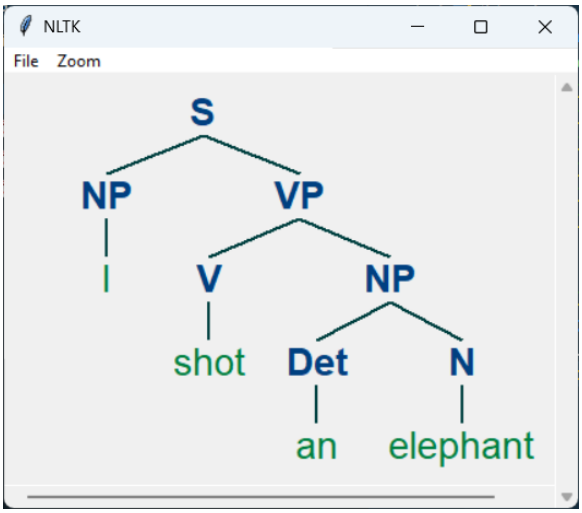
3. **Define a grammar(CFG) and generate parse tree results using NLTK.**

Code

```
import nltk
from nltk.tree import *

# Function to define grammar and parse result
def definegrammar_parseresult():
    # Define a context-free grammar (CFG)
    Grammar = nltk.CFG.fromstring("""
        S -> NP VP
        PP -> P NP
        NP -> Det N | Det N PP | 'I'
        VP -> V NP | VP PP
        Det -> 'an' | 'my'
        N -> 'elephant' | 'pajamas'
        V -> 'shot'
        P -> 'in'
    """)

    # Tokenized sentence to be parsed
    sent = "I shot an elephant".split()

    # Initialize the parser with the grammar
    parser = nltk.ChartParser(Grammar)

    trees = list(parser.parse(sent))  # Generate parse trees

    # Display parse trees
    if not trees:
        print("No parse tree found.")
    else:
        print("Parse Tree(s):\n")
        for tree in trees:
            tree.pretty_print()   # Print tree in text form
            tree.draw()           # Opens graphical tree window

# Run the function
definegrammar_parseresult()
```

Output

```
Parse Tree(s):

          S
  ________|___
 |            VP
 |     ________|___
 |    |            NP
 |    |         ___|_____
 NP   V        Det        N
 |    |         |         |
 I   shot       an     elephant
```



4. **Using a porter stemmer library derives derivational and inflectional morphemes.**

Code

```
from nltk.stem import PorterStemmer # Import the PorterStemmer class

# Define input words for demonstration
word = "unexpected" # Example word to stem
text = "disagreement" # Example word with derivational morphemes
text1 = "disagree" # Example word without derivational morphemes
text2 = "agreement" # Example word for comparison
text3 = "quirkiness" # Word to analyze for stemming
text4 = "historical" # Word with historical suffix
text5 = "cannonical" # Word with canonical suffix
text6 = "happiness" # Word with derivational morphemes
text7 = "unkind" # Word with derivational morphemes
text8 = "dogs" # Word with inflectional morphemes
text9 = "expected" # Word with inflectional morphemes

# Define lists for derivational and inflectional morphemes
words_derv = ["happiness", "unkind"] # Words with derivational morphemes
words_infle = ["dogs", "expected"] # Words with inflectional morphemes
words = ["unexpected", "disagreement", "disagree", "agreement",
         "quirkiness", "historical", "cannonical", "happiness",
         "unkind", "dogs", "expected"]

# Function to demonstrate stemming using PorterStemmer
def stemmer_porter():
    port = PorterStemmer() # Initialize the Porter stemmer
    # Process derivational morphemes
    print("Derivational Morphemes")
    print(" ".join([port.stem(i) for i in text6.split()])) # Stem
'happiness'
    print(" ".join([port.stem(i) for i in text7.split()])) # Stem
'unkind'

    # Process inflectional morphemes
    print("\nInflectional Morphemes")
    print(" ".join([port.stem(i) for i in text8.split()])) # Stem 'dogs'
    print(" ".join([port.stem(i) for i in text9.split()])) # Stem
```

```
'expected'

    # Demonstrate stemming on various examples
    print("\nSome Examples")
    print(" ".join([port.stem(i) for i in word.split()])) # Stem
'unexpected'
    print(" ".join([port.stem(i) for i in text.split()])) # Stem
'disagreement'
    print(" ".join([port.stem(i) for i in text1.split()])) # Stem
'disagree'
    print(" ".join([port.stem(i) for i in text2.split()])) # Stem
'agreement'
    print(" ".join([port.stem(i) for i in text3.split()])) # Stem
'quirkiness'
    print(" ".join([port.stem(i) for i in text4.split()])) # Stem
'historical'
    print(" ".join([port.stem(i) for i in text5.split()])) # Stem
'cannonical'

# Run the function if the script is executed directly
if __name__ == "__main__":
    stemmer_porter()
```

Output

```
Derivational Morphemes
happi
unkind

Inflectional Morphemes
dog
expect

Some Examples
unexpect
disagr
disagre
agreement
quirki
histor
cannon
```

Practical 4: Illustrate part of speech tagging.

---

1. **Part of speech Tagging and chunking of user defined text.**

!pip install spacy
!python -m spacy download en_core_web_sm

Code

```
import spacy # Import spaCy library
nlp = spacy.load("en_core_web_sm") # Load the small English language
model

# Process the text
text = "The intelligent student completed the assignment and submitted
the project on time."
doc = nlp(text)

# Extract and print noun phrases (chunks) from the text
print("Noun Phrases:", [chunk.text for chunk in doc.noun_chunks]) #
Extract noun phrases

# Extract and print verbs (lemmas) from the text
print("Verbs:", [token.lemma_ for token in doc if token.pos_ == "VERB"])
```

Output

```
Noun Phrases: ['The intelligent student', 'the assignment', 'the project',
'time']
Verbs: ['complete', 'submit']
```

2. Named Entity recognition of user defined text.

Code

```
import spacy # Import spaCy library
nlp = spacy.load("en_core_web_sm") # Load the small English language
model

# Take user-defined text as input
print("Enter any word from the following statement:")
user_text = input("Sachin Tendulkar played cricket for India and worked
with BCCI in Mumbai in 2015. ----> ")
doc = nlp(user_text) # Process the user input text

# Print all named entities in the text
print("\nNamed Entities: ")
for ent in doc.ents:
  print(f"Text: {ent.text}, Label: {ent.label_}") # Print named entities
and their labels

# Print explanations for the named entity labels
print("\nLabel Explanation:")
for ent in doc.ents:
  print(f"{ent.text}: {spacy.explain(ent.label_)}") # Explain each
entity label
```

Output

```
Enter any word from the following statement:
Sachin Tendulkar played cricket for India and worked with BCCI in Mumbai in 2015. ----> India

Named Entities:
Text: India, Label: GPE

Label Explanation:
India: Countries, cities, states
```

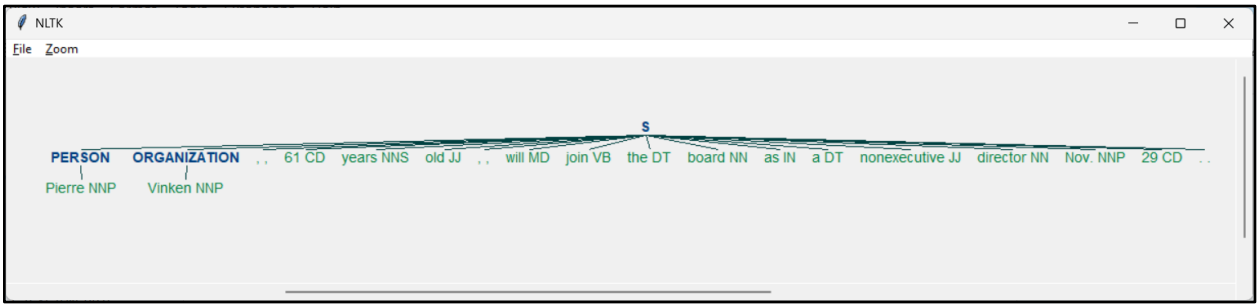
```
Enter any word from the following statement:
Sachin Tendulkar played cricket for India and worked with BCCI in Mumbai in 2015. ----> 2015

Named Entities:
Text: 2015, Label: DATE

Label Explanation:
2015: Absolute or relative dates or periods
```

3. **Named Entity recognition with diagram using NLTK corpus – treebank.**

Code

```
import nltk
from nltk.corpus import treebank
from nltk import pos_tag, ne_chunk
from nltk.tree import Tree
# Ensure necessary NLTK datasets are downloaded
nltk.download('treebank')
nltk.download('maxent_ne_chunker')
nltk.download('words')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('maxent_ne_chunker_tab')

# Step 1: Load sentences from the treebank corpus
sentence = treebank.sents()[0] # Use the first sentence as an example

# Step 2: Part-of-Speech tagging
pos_tagged_sentence = pos_tag(sentence)

# Step 3: Perform Named Entity Recognition
named_entity_tree = ne_chunk(pos_tagged_sentence)

# Step 4: Display the named entity tree
print(named_entity_tree)

# Step 5: Draw the named entity tree (requires GUI support)
named_entity_tree.draw()
```

Output



```
(S
  (PERSON Pierre/NNP)
  (ORGANIZATION Vinken/NNP)
  ,/,
  61/CD
  years/NNS
  old/JJ
  ,/,
  will/MD
  join/VB
  the/DT
  board/NN
  as/IN
  a/DT
  nonexecutive/JJ
  director/NN
  Nov./NNP
  29/CD
  ./.)
```

Practical 5: Stop Words.

---

1. Handling stopwords.

!pip install stopwords

```
!pip install stopwords
```

```
Collecting stopwords
  Downloading stopwords-1.0.2-py2.py3-none-any.whl.metadata (2.3 kB)
Downloading stopwords-1.0.2-py2.py3-none-any.whl (37 kB)
Installing collected packages: stopwords
Successfully installed stopwords-1.0.2
```

Code

```
import nltk # Importing NLTK library

nltk.download('punkt_tab') # Downloading tokenization resources
nltk.download('stopwords') # Downloading NLTK's stopword list

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# Defining a sample text
word_quote = "Good morning! My name is Shivam Vishwkarma Roll no 6709."
words_in_quote = word_tokenize(word_quote, "english") # Tokenizing the
text
print(words_in_quote) # Printing the tokenized words

stop_words = set(stopwords.words("english")) # Getting English stopwords
filtered_list = [] # Initializing list to store words after stopword
removal

# Iterating through words to filter out stopwords
for word in words_in_quote:
  if word.casefold() not in stop_words: # If word is not a stopword
    filtered_list.append(word) # Adding non-stopword to the list
    print(filtered_list) # Printing the filtered list
```

Output

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
['Good', 'morning', '!', 'My', 'name', 'is', 'Shivam', 'Vishwkarma',
'Roll', 'no', '6709', '.']
['Good']
['Good', 'morning']
['Good', 'morning', '!']
['Good', 'morning', '!', 'name']
['Good', 'morning', '!', 'name', 'Shivam']
['Good', 'morning', '!', 'name', 'Shivam', 'Vishwkarma']
['Good', 'morning', '!', 'name', 'Shivam', 'Vishwkarma', 'Roll']
['Good', 'morning', '!', 'name', 'Shivam', 'Vishwkarma', 'Roll', '6709']
['Good', 'morning', '!', 'name', 'Shivam', 'Vishwkarma', 'Roll', '6709',
'.']
```

2. Using NLTK Adding or Removing Stop Words in NLTK's Default Stop Word.

Code

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# Download required data (run once in Colab)
nltk.download("stopwords")

# example sentence
text = "Shivam is not happy with the invoice."

# Convert sentence into words
words = word_tokenize(text)
print("Original Words:", words)

# load default English stopwords
stop_words = stopwords.words("english")

# Remove default stopwords
filtered_words = [word for word in words if word.lower() not in
stop_words]
print("\nAfter Removing Default Stopwords:", filtered_words)

# Add a new stopword (example: 'invoice')
stop_words.append("invoice")

filtered_words_added = [word for word in words if word.lower() not in
stop_words]
print("\nAfter Adding 'invoice' to Stopwords:", filtered_words_added)

# Remove an existing stopword (example: 'not')
print("\nStop word before removing 'not': " , stop_words)

if "not" in stop_words:
    stop_words.remove("not")

filtered_words_removed = [word for word in words if word.lower() not in
```

```
stop_words]

print("\nAfter Removing 'not' from Stopwords:", filtered_words_removed)
```

Output

```
Original Words: ['Shivam', 'is', 'not', 'happy', 'with', 'the', 'invoice',
'.']

After Removing Default Stopwords: ['Shivam', 'happy', 'invoice', '.']

After Adding 'invoice' to Stopwords: ['Shivam', 'happy', '.']

After Removing 'not' from Stopwords: ['Shivam', 'not', 'happy', '.']

[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

3. Using Spacy Adding and Removing Stop Words in Default Spacy Stop Words List

Code

```
import spacy  # Importing spaCy library
from nltk.tokenize import word_tokenize  # Importing word tokenizer
from nltk.corpus import stopwords    # Importing stopwords from corpus
import string     # Importing string module for punctuation removal

# Load SpaCy English model
sp = spacy.load('en_core_web_sm')

# Get default stopwords from spaCy
all_stopwords = sp.Defaults.stop_words
print("stopwords (20):", all_stopwords)

# Add comma as stopword
all_stopwords.add(",")

# Sample text
text = "Shivam is not happy with the invoice."
print("\nText Example: ", text)

# Tokenize the text
text_tokens = word_tokenize(text)

# Print tokens that are stopwords (using NLTK stopwords)
tokens_with_sw = [
    word for word in text_tokens
    if word in stopwords.words('english')
]

print("\nToken with stopwords:", tokens_with_sw)

# Remove "not" from stopword set if present
if 'not' in all_stopwords:
    all_stopwords.remove('not')

# Remove stopwords from text
tokens_without_sw = [
```

```
    word for word in text_tokens
    if word not in all_stopwords
]

print("Token without stopwords:", tokens_without_sw)

# Remove punctuation from tokens
tokens_without_punctuation = [
    word for word in tokens_without_sw
    if word not in string.punctuation
]

print("Tokens without punctuation:", tokens_without_punctuation)
```

Output

```
stopwords (20): ["'s", 'after', 'except', 'now', 'three', 'otherwise',
'somewhere', 'none', 'no', 'n't', 'side', '`d', 'becoming', 'hereafter',
"'ve", 'there', 'through', 'into', 'between', 'seeming']

Text Example:  Shivam is not happy with the invoice.

Token with stopwords: ['is', 'not', 'with', 'the']
Token without stopwords: ['Shivam', 'not', 'happy', 'invoice', '.']
Tokens without punctuation: ['Shivam', 'not', 'happy', 'invoice']
```

Practical 6: Hyponyms and Hypernyms.

---

1. **Write a program using python to find synonyms and antonyms of word "active" using Wordnet.**

Code

```
import nltk
nltk.download('wordnet')
from nltk.corpus import wordnet

# Find Synsets (Synonyms)
print("Synsets (Synonyms) of 'active'")
synsets = wordnet.synsets("active")
print(synsets)

# Find Antonyms
print("\nAntonyms of 'active'")
antonyms = wordnet.lemma('active.a.01.active').antonyms()
print(antonyms)
```

Output

```
Synsets (Synonyms) of 'active'
[Synset('active_agent.n.01'), Synset('active_voice.n.01'),
Synset('active.n.03'), Synset('active.a.01'), Synset('active.s.02'),
Synset('active.a.03'), Synset('active.s.04'), Synset('active.a.05'),
Synset('active.a.06'), Synset('active.a.07'), Synset('active.s.08'),
Synset('active.a.09'), Synset('active.a.10'), Synset('active.a.11'),
Synset('active.a.12'), Synset('active.a.13'), Synset('active.a.14')]

Antonyms of 'active'
[Lemma('inactive.a.02.inactive')]
```

2. **Compare two nouns.**

Code

```
import nltk
nltk.download('wordnet')
from nltk.corpus import wordnet

# Get synsets of both nouns
syn1 = wordnet.synsets("basketball")
syn2 = wordnet.synsets("volleyball")

# Compare each synset using Path Similarity
print("Comparing 'basketball' and 'volleyball'")

for s1 in syn1:      # Loop through synsets of "basketball"
    for s2 in syn2:  # Loop through synsets of "volleyball"

        print("\nPath Similarity Between:")

        # Print details of first synset
        print("Noun 1:", s1,
              "(", s1.pos(), ")",
              "[", s1.definition(), "]")

        # Print details of second synset
        print("Noun 2:", s2,
              "(", s2.pos(), ")",
              "[", s2.definition(), "]")

        # Calculate and print similarity
        print("Path Similarity is:", s1.path_similarity(s2))
```

Output

```
Comparing 'basketball' and 'volleyball'

Path Similarity Between:
Noun 1: Synset('basketball.n.01') ( n ) [ a game played on a court by two
opposing teams of 5 players; points are scored by throwing the ball
through an elevated horizontal hoop ]
Noun 2: Synset('volleyball.n.01') ( n ) [ a game in which two teams hit an
inflated ball over a high net using their hands ]
Path Similarity is: 0.3333333333333333

Path Similarity Between:
Noun 1: Synset('basketball.n.01') ( n ) [ a game played on a court by two
opposing teams of 5 players; points are scored by throwing the ball
through an elevated horizontal hoop ]
Noun 2: Synset('volleyball.n.02') ( n ) [ an inflated ball used in playing
volleyball ]
Path Similarity is: 0.05263157894736842

Path Similarity Between:
Noun 1: Synset('basketball.n.02') ( n ) [ an inflated ball used in playing
basketball ]
Noun 2: Synset('volleyball.n.01') ( n ) [ a game in which two teams hit an
inflated ball over a high net using their hands ]
Path Similarity is: 0.05263157894736842

Path Similarity Between:
Noun 1: Synset('basketball.n.02') ( n ) [ an inflated ball used in playing
basketball ]
Noun 2: Synset('volleyball.n.02') ( n ) [ an inflated ball used in playing
volleyball ]
Path Similarity is: 0.3333333333333333
```

3. **Study lemmas, hyponyms, hypernyms.**

Code

```
import nltk
nltk.download('wordnet')
from nltk.corpus import wordnet

# Synsets and Lemmas
print("Synsets of 'computer'")
print(wordnet.synsets("computer"))

print("\nLemma Names of 'computer.n.01'")
print(wordnet.synset("computer.n.01").lemma_names())

print("\nAll Synsets and Their Lemmas")
for e in wordnet.synsets("computer"):
    print(f"\n{e} --> {e.lemma_names()}")

    # Lemmas of specific synset
    print("Lemmas of computer.n.01:",
          wordnet.synset('computer.n.01').lemmas())

    # Synset from lemma
    print("Synset of 'computer.n.01.computing_device':",
          wordnet.lemma('computer.n.01.computing_device').synset())

    # Lemma name
    print("Lemma name:",
          wordnet.lemma('computer.n.01.computing_device').name())

# Hyponyms
print("\n\nHyponyms of 'computer.n.01'")

syn = wordnet.synset('computer.n.01')

# Print hyponym synsets
print("Hyponym Synsets:")
print(syn.hyponyms())
```

```
# Print hyponym lemma names
print("\nHyponym Lemma Names:")
print([lemma.name()
       for synset in syn.hyponyms()
       for lemma in synset.lemmas()])

# Hypernyms (Common Hypernym Example)
print("\nHypernym Example (Vehicle vs Car)")

vehicle = wordnet.synset('vehicle.n.01')
car = wordnet.synset('car.n.01')

print("Lowest Common Hypernyms:")
print(car.lowest_common_hypernyms(vehicle))
```

Output

```
Synsets of 'computer'
[Synset('computer.n.01'), Synset('calculator.n.01')]

Lemma Names of 'computer.n.01'
['computer', 'computing_machine', 'computing_device', 'data_processor',
'electronic_computer', 'information_processing_system']

All Synsets and Their Lemmas

Synset('computer.n.01') --> ['computer', 'computing_machine',
'computing_device', 'data_processor', 'electronic_computer',
'information_processing_system']
Lemmas of computer.n.01: [Lemma('computer.n.01.computer'),
Lemma('computer.n.01.computing_machine'),
Lemma('computer.n.01.computing_device'),
Lemma('computer.n.01.data_processor'),
Lemma('computer.n.01.electronic_computer'),
Lemma('computer.n.01.information_processing_system')]
Synset of 'computer.n.01.computing_device': Synset('computer.n.01')
Lemma name: computing_device

Synset('calculator.n.01') --> ['calculator', 'reckoner', 'figurer',
'estimator', 'computer']
Lemmas of computer.n.01: [Lemma('computer.n.01.computer'),
Lemma('computer.n.01.computing_machine'),
Lemma('computer.n.01.computing_device'),
```

```
Lemma('computer.n.01.data_processor'),
Lemma('computer.n.01.electronic_computer'),
Lemma('computer.n.01.information_processing_system')]
Synset of 'computer.n.01.computing_device': Synset('computer.n.01')
Lemma name: computing_device


Hyponyms of 'computer.n.01'
Hyponym Synsets:
[Synset('node.n.08'), Synset('digital_computer.n.01'),
Synset('predictor.n.03'), Synset('web_site.n.01'), Synset('server.n.03'),
Synset('analog_computer.n.01'), Synset('turing_machine.n.01'),
Synset('number_cruncher.n.02'), Synset('home_computer.n.01'),
Synset('pari-mutuel_machine.n.01')]

Hyponym Lemma Names:
['node', 'client', 'guest', 'digital_computer', 'predictor', 'web_site',
'website', 'internet_site', 'site', 'server', 'host', 'analog_computer',
'analogue_computer', 'Turing_machine', 'number_cruncher', 'home_computer',
'pari-mutuel_machine', 'totalizer', 'totaliser', 'totalizator',
'totalisator']

Hypernym Example (Vehicle vs Car)
Lowest Common Hypernyms:
[Synset('vehicle.n.01')]
```

Practical 7: Tagged Corpora

---

1. **Study of tagged corpora with methods like tagged_sents, tagged_words.**

Code

```
import nltk
nltk.download('brown')
nltk.download('universal_tagset')

from nltk.corpus import brown

print("Tagged Words")
# Show first 20 tagged words
print(brown.tagged_words(tagset='universal')[:20])

print("\nTagged Sentences")
# Show first 3 tagged sentences
for sent in brown.tagged_sents(tagset='universal')[:2]:
    print(sent)
```

Output

```
Tagged Words

[('The', 'DET'), ('Fulton', 'NOUN'), ('County', 'NOUN'), ('Grand', 'ADJ'),
('Jury', 'NOUN'), ('said', 'VERB'), ('Friday', 'NOUN'), ('an', 'DET'),
('investigation', 'NOUN'), ('of', 'ADP'), ("Atlanta's", 'NOUN'),
('recent', 'ADJ'), ('primary', 'NOUN'), ('election', 'NOUN'), ('produced',
'VERB'), ('``', '.'), ('no', 'DET'), ('evidence', 'NOUN'), ("''", '.'),
('that', 'ADP')]

Tagged Sentences

[('The', 'DET'), ('Fulton', 'NOUN'), ('County', 'NOUN'), ('Grand', 'ADJ'),
('Jury', 'NOUN'), ('said', 'VERB'), ('Friday', 'NOUN'), ('an', 'DET'),
('investigation', 'NOUN'), ('of', 'ADP'), ("Atlanta's", 'NOUN'),
('recent', 'ADJ'), ('primary', 'NOUN'), ('election', 'NOUN'), ('produced',
'VERB'), ('``', '.'), ('no', 'DET'), ('evidence', 'NOUN'), ("''", '.'),
('that', 'ADP'), ('any', 'DET'), ('irregularities', 'NOUN'), ('took',
'VERB'), ('place', 'NOUN'), ('.', '.')]
```

```
[('The', 'DET'), ('jury', 'NOUN'), ('further', 'ADV'), ('said', 'VERB'),
('in', 'ADP'), ('term-end', 'NOUN'), ('presentments', 'NOUN'), ('that',
'ADP'), ('the', 'DET'), ('City', 'NOUN'), ('Executive', 'ADJ'),
('Committee', 'NOUN'), (',', '.'), ('which', 'DET'), ('had', 'VERB'),
('over-all', 'ADJ'), ('charge', 'NOUN'), ('of', 'ADP'), ('the', 'DET'),
('election', 'NOUN'), (',', '.'), ('``', '.'), ('deserves', 'VERB'),
('the', 'DET'), ('praise', 'NOUN'), ('and', 'CONJ'), ('thanks', 'NOUN'),
('of', 'ADP'), ('the', 'DET'), ('City', 'NOUN'), ('of', 'ADP'),
('Atlanta', 'NOUN'), ("'''", '.'), ('for', 'ADP'), ('the', 'DET'),
('manner', 'NOUN'), ('in', 'ADP'), ('which', 'DET'), ('the', 'DET'),
('election', 'NOUN'), ('was', 'VERB'), ('conducted', 'VERB'), ('.', '.')]
```

2. **Write a program to find the most frequent noun tags.**

Code

```
# Importing necessary modules
import nltk
from collections import defaultdict

# Download required dataset
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('punkt')

text = "Shivam bought books, laptops, and notebooks from the market."

# Tokenization
text = nltk.word_tokenize(text)

# POS Tagging
tagged = nltk.pos_tag(text)  # (word, POS-tag)

print("Tagged Words:", tagged)

# Extract Noun Words
addNounWords = []
count = 0

for words in tagged:
    val = tagged[count][1]  # POS tag

    # Check for noun tags
    if (val == 'NN' or val == 'NNS' or val == 'NNPS' or val == 'NNP'):
        addNounWords.append(tagged[count][0])

    count += 1

print("\nNoun Words:", addNounWords)

# Count Frequency of Nouns
temp = defaultdict(int)
```

```
for sub in addNounWords:
    for wrd in sub.split():
        temp[wrd] += 1

# Find Most Frequent Noun
res = max(temp, key=temp.get)
print("\nWord with maximum frequency:", str(res))
```

Output

```
Tagged Words: [('Shivam', 'NNP'), ('bought', 'VBD'), ('books', 'NNS'),
(',', ','), ('laptops', 'NNS'), (',', ','), ('and', 'CC'), ('notebooks',
'RB'), ('from', 'IN'), ('the', 'DT'), ('market', 'NN'), ('.', '.')]

Noun Words: ['Shivam', 'books', 'laptops', 'market']

Word with maximum frequency: Shivam
```

3. Map Words to Properties Using Python Dictionaries.

Code

```
# Creating a dictionary
thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}

# Print entire dictionary
print("Dictionary:", thisdict)

# Access value using key
print("Brand:", thisdict["brand"])

# Print number of key-value pairs
print("Length of dictionary:", len(thisdict))

# Print data type
print("Type of dictionary:", type(thisdict))
```

Output

```
Dictionary: {'brand': 'Ford', 'model': 'Mustang', 'year': 1964}
Brand: Ford
Length of dictionary: 3
Type of dictionary: <class 'dict'>
```

Practical 8: NLP libraries for Hindi language.

---

1. Word tokenization in Hindi.

Installing the 'inltk' package for Natural Language Processing (NLP) in Indian languages
!pip install inltk

Installing and downloading two spaCy models:

- xx_ent_wiki_sm - A multilingual model for named entity recognition (NER) across several languages, including Hindi
- en_core_web_md - A medium-sized model for English (specifically for tasks like word vectors and semantic similarity)

!python -m spacy download xx_ent_wiki_sm
!python -m spacy download en_core_web_md

Code

```
# Filename: hindi_word_tokenization_spacy.py

# Import spaCy library
import spacy

# Load spaCy multilingual model (better for Hindi)
nlp = spacy.load('xx_ent_wiki_sm')
nlp = spacy.load('en_core_web_md') # Load the English spaCy model
(replacing previous load)

# Sample Hindi text for processing
hindi_text = "यह एक उदाहरण वाक्य है। इसमें कई शब्द हैं।"

# Process the Hindi text
doc = nlp(hindi_text)

# Tokenization
tokens = [token.text for token in doc]
print("Tokenized Words: ", tokens)

# Function to calculate similarity
```

```
def get_similarity(text1, text2):
    doc1 = nlp(text1)
    doc2 = nlp(text2)

    similarity = doc1.similarity(doc2)
    return similarity

# Example Hindi texts for similarity comparison
text1 = "यह एक उदाहरण वाक्य है।"
text2 = ""

# Calculate similarity
output = get_similarity(text1, text2)
print("\nSimilarity Output:", output)
```

Output

Tokenized Words: ['यह', 'एक', 'उदाहरण', 'वाक्य', 'है', '।', 'इसमें', 'कई', 'शब्द', 'हैं', '।']

```
Similarity Output: 0.0
```

Installing the 'transformers' package to work with pre-trained transformer models
!pip install transformers

Code

```
# Importing relevant modules from the 'transformers' library
from transformers import AutoTokenizer, AutoModel
import torch   # PyTorch for tensor operations

# Load Multilingual BERT Model -Supports multiple languages including
Hindi
model_name = "bert-base-multilingual-cased"

tokenizer = AutoTokenizer.from_pretrained(model_name)   # Tokenizer
model = AutoModel.from_pretrained(model_name)           # Pre-trained
model

# Function: Get Sentence Embedding
def get_sentence_embedding(sentence):

    # Tokenize and convert to tensor
    inputs = tokenizer(
        sentence,
        return_tensors='pt',
        padding=True,
        truncation=True,
        max_length=128
    )

    # Disable gradients for inference
    with torch.no_grad():
        outputs = model(**inputs)

    # Return mean of last hidden state (sentence embedding)
    return outputs.last_hidden_state.mean(dim=1).squeeze()

# Function: Calculate Cosine Similarity
def get_similarity(text1, text2):

    # Get embeddings
```

```
    emb1 = get_sentence_embedding(text1)
    emb2 = get_sentence_embedding(text2)

    # Cosine similarity
    similarity = torch.cosine_similarity(
        emb1.unsqueeze(0),
        emb2.unsqueeze(0)
    ).item()

    return similarity

# Example Hindi Sentences
text1 = "यह एक उदाहरण वाक्य है।"
text2 = "यह एक"

# Calculate similarity
similarity = get_similarity(text1, text2)
print(f"Sentence Similarity: {similarity}")
```

Output

```
Loading weights: 100%|□□□
```